

Last modified on

Sports Data Soccer API - In-Venue Enriched Events Feed (MA36)

Get all events in a game - including the player, team, event, type, player coordinates, pass, pressure, threat, time (minute and second) - and qualifiers for each action, enriched with latest Opta Vision metrics based on venue data.

Overview

The Soccer **Enriched Soccer Event In-Venue** is a time-coded feed that lists all individual events and player actions in a match. Each incidence in the feed includes a contestant (team), event, type, and timestamp for each action. Every event has a series of qualifiers describing it. The enriched events will also have line breaking pass, pressure, threat, pass options and pass predictions related details. You will find explanations of every possible event and qualifier that could be returned in the response in our complete lists of **Events Types** and **Qualifier Types**. See the **Appendix** to put this into context and get information about the following: **Pitch coordinates and direction of play** | **Goalmouth coordinates** | **Pitch zones** | **Playing positions** | **Formations**

For more details refer: <https://www.theanalyst.com/eu/2021/09/expected-pass-completion-explained/>

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY							
	TIER 15	TIER 12+14	TIER 4	TIER 13	TIER 8+9+10+11	TIER 6+7	TIER 5	TIER 1+2+3
Updated on every stop in play during the game, including for the following types of events: goal, shot at goal, assist, free kick, offside given, cards, start of the half, end of the half, corner, substitution, lineup	See Tier 13	Post-match	n/a	No more frequently than one URL poll every 30 seconds for range: <KO - 1h; KO + 4h> Up to 1-hour interval for range: <KO + 4h; KO + 8h>	n/a	n/a	n/a	n/a

UPDATED	POLLING FREQUENCY							
	TIER 15	TIER 12+14	TIER 4	TIER 13	TIER 8+9+10+11	TIER 6+7	TIER 5	TIER 1+2+3
Note: There will always be a gap of at least 30 seconds between files unless a goal is scored				Up to 24-hour interval 8h+ of match ending, lasting a maximum of 7 days				

Enriched Soccer Events In-Venue (MA36) Guide

USAGE	FEED
GET enriched soccer events (in venue) list by fixture UUID.	/soccerdata/invenueevents/{outletAuthKey}/{f{queryParameters}}
GET enriched soccer events (in venue) list by fixture UUID inline URL path.	/soccerdata/invenueevents/{outletAuthKey}?{f{queryParameters}}

If your request URL is as follows (does not contain a fixture UUID), the feed returns an error:
/soccerdata/invenueevents/{outletAuthKey}

How can I get fixture and other entity UUIDs?

Use the [Tournament Calendar \(OT2\)](#) or [Fixtures and Results \(MA1\)](#) feeds to get fixture metadata, including its unique ID (UUI), linked contestants, competition, tournament calendar and other entities.

Query Parameters And Example Request Parameters

You must query the `/invenueevents` feed by a fixture UUID. This feed also makes use of the [Global query parameters](#), and you can combine them with the following query parameters (some of which can be used in conjunction with each other.)

Legacy IDs: You can use the `fx` parameter to filter by legacy IDs - only one ID is supported (you cannot pass multiple IDs in a comma-separated list).

Making successful requests: In our guides, we use HTTPS syntax for our example `GET` requests (rather than `cURL`, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You MUST format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the [main guide](#) and [API Overview and Authorisation guide](#).

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` this must be after the Outlet Authentication Key (and endpoint, if applicable). for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format). If required, it should also include an entity UUID (such as a fixture/match UUID).
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by

the **&** (ampersand) operator. Values must be separated by the correct operator, for example **&** (multiple values, 'AND'), or - if a parameter supports it - **,** (multiple values, 'OR') or **!** ('NOT').

- Use the required **_** (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces **{ }** from your calls (we include these as an example only).
- If using the JSONP format (**_fmt=jsonp**) you MUST use it in combination with the callback parameter (**_clbk**) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
fixture uuid	GET Enriched Soccer Events (in venue) list by fixture UUID inline URL path. GET http://api.performfeeds.com/soccerdata/invenueevents/{outletAuthKey}/{fixtureUuid?_fmt={dataFormat}&_rt=b
fx	GET Enriched Soccer Events (in venue) list by fixture UUID. GET http://api.performfeeds.com/soccerdata/invenueevents/{outletAuthKey}?{fx=fixtureUuid&_fmt={dataFormat}&_rt=b
playingstyles	GET Enriched Soccer Events (in venue) list by fixture UUID and playingstyles GET http://api.performfeeds.com/soccerdata/remoteevents/{outletAuthKey}?fx={fixtureUuid&playingstyles={yes no}&_fmt={dataFormat}&_rt=b Note:Two new nodes <marking> and <lineBreaking> inside parent node <passOption> → <player>, which will only be returned if the API is queried using parameter playingstyles. The parent node <marking>, that contains multiple child nodes <opponent>, which provides information related to pass target quality. The data will only be present in the response if the feed is queried using playingstyles=yes query parameter.

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- [XML](#)
- [JSON](#)

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

- The root element can be `inVenueEvents` or `matchInfo` - see the table below.
- In the scenario where the match has not yet been played (for example, it is a scheduled fixture to be played in the future), not all attributes listed here will be present and we may provide placeholder values for some attributes. For example, the scores, winner, contestant name, position, and ID may not be known (the contestant `id` attribute may also not be returned). However, the feed will be updated with this information once it is available.

▶<inVenueEvents> (XML only)

▶<matchInfo>

▶<description>

▶<sport>

▶<ruleset>

▶<competition>

▶<country>

▶<tournamentCalendar>

▶<stage>

▶<contestants> (XML only)

▶<contestant>

▶<country>

▶<venue>

▶<liveData>

▶<matchDetails>

▶<periods> (XML only)

▶<period>

▶<suspensions>

▶<suspension>

▶<scores>

▶<ht>

▶<ft>

▶<et>

- ▶<pen>
- ▶<total>
- ▶<totalUnconfirmed>
- ▶<events> (XML only)
- ▶<event>
- ▶<qualifier>
- ▶<shotDistance>
- ▶<reception>
- ▶<player>
- ▶<pressureReceived>
- ▶<opponent>
- ▶<lineBreaking>
- ▶<numberOfLinesBroken>
- ▶<lastLineBroken>
- ▶<mainActor>
- ▶<lineBreakingPass>
- ▶<linesBroken>
- ▶<lastLineBroken>
- ▶<pressure>
- ▶<pressureReceived>
- ▶<player>
- ▶<passOption>
- ▶<player>
- ▶<predictions>
- ▶<expectedPassReceiver>
- ▶<expectedPassCompletion>
- ▶<expectedThreat>
- ▶<passOptionQuality>

▶<marking>
 ▶<opponent>
 ▶<lineBreaking>
 ▶<numberOfLinesBroken>
 ▶<lastLineBroken>
 ▶<passOptionQuality>
 ▶<passTarget>
 ▶<player>
 ▶<predictions>
 ▶<expectedPassReceiver>
 ▶<expectedPassCompletion>
 ▶<expectedThreat>
 ▶<passOptionQuality>
 ▶<marking>
 ▶<opponent>
 ▶<xThreat>

Event Types

Events are delivered as an array (under `events` for XML or `events` for JSON). When an event occurs during a match, it is assigned a unique Event ID. For each unique Event ID, there is an associated Event Type ID (`typeId`) which classifies the general event type that has occurred. It is common for an event instance to be accompanied by a Qualifier array (under `qualifier`). Each Qualifier has an ID and a Qualifier ID (`qualifierId`) and value - these explain and add depth and further information about the event. However, it is not mandatory (some events will not have qualifiers).

Use this table to look-up an Event Type ID and its description. You should also cross-reference any related Qualifier ID values with the [Qualifier Types](#) list.

Event Definitions

We have a long-established, consistent list of 'Event Definitions' in football that are adhered to across all data collection centres. You can find the most up-to-date definitions in the [Opta Glossary](#) (opens the Opta website), where we maintain an up-to-date list of the main football events logged by Opta.

▶ **SHOW/HIDE: event typeId | name | description | associated qualifierId values**

Qualifier Types

Qualifiers are linked to Event Types and provide more detailed information about the specific event that occurred. Each event can have a series of qualifiers describing it.

The Event Type ID classifies the general event type (such as a Pass event: `"typeId": 1`).

The Qualifier array provides further details (such as pitch coordinates or the type of Pass event) - it contains the following:

- `qualifierId` - this represents the specific event. For example, the type of Pass event, such as a 'Through Ball' (`"qualifierId": 4`).
- `value` - contains any related information. For example, pitch coordinates for the pass start point and pass end point.

Use this table to look-up a Qualifier ID and its description. You should also cross-reference any related Event Type ID values with the [Event types list](#), and check the [Appendix](#) to get more information about the following: [Pitch coordinates and direction of play](#) | [Goalmouth coordinates](#) | [Pitch zones](#) | [Playing positions](#) | [Formations explained](#) | [Kit IDs](#)

► [SHOW/HIDE: qualifierId | name | associated event typeId values](#)

Useful Queries

► [SHOW/HIDE: Type_id | outcome | qualifier_id | Extra info | Excluding Events These Qualifiers:](#)

Appendix

This section helps give further context to some Events and their Qualifiers, such as pitch coordinates and other data. Click on the buttons below to view each section.

Appendix 1 - Pitch Coordinates And Direction Of Play

► [SHOW/HIDE](#)

Appendix 2 - Goalmouth Coordinates

► [SHOW/HIDE](#)

Appendix 3 - Pitch Zones

► [SHOW/HIDE](#)

Appendix 4 - Playing Positions

► [SHOW/HIDE](#)

Appendix 5 - Formations Explained

► [SHOW/HIDE](#)

Appendix 6 - Kit IDs

► [SHOW/HIDE](#)

Appendix 7 - Coordinates Transformation

►SHOW/HIDE